Chapter 03 대칭키 암호(1)

Venture 1st
호서대학교

목차

3.1 개요
3.2 스트림 암호
3.3 블록 암호
3.4 무결성
3.5 양자 컴퓨터와 대칭 암호
3.6 요약

3.1 개요

➢ 스트림 암호(stream cipher)

- 전통적인 일회성 암호 시스템과 개념이 비슷함
- 일회성 암호와 같은 방식으로 키를 긴 비트열로 늘여서 사용
- 일회성 암호처럼 섀넌의 원칙 중 혼돈 이론만 적용
- 예) A5/1, RC4

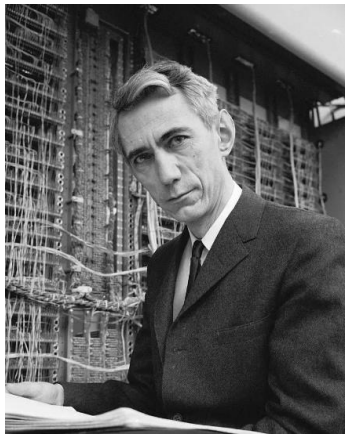
➢ 블록 암호(block cipher)

- 고전 코드북 암호의 현대 버전
- 혼돈과 확산의 원칙을 모두 사용
- 예) DES, AES, TEA, SEED

참고

➢ 클로드 엘우드 섀넌(Claude Elwood Shannon)

- 미국의 수학자이자 컴퓨터 과학자
- 부울 대수와 전자회로를 결합시켜 디지털 논리회로 이론 창시
- One-Time Pad의 안전성 증명



- "A Mathematical Theory of Communication(1948)" 논문 발표
  - 정보이론의 아버지라 불림

- "Communication Theory of Secrecy Systems(1949)" 논문 발표
  - 암호 시스템 설계의 두가지 원칙인 혼돈(Confusion)과 확산(Diffusion) 이론 제시

참고 - 혼돈(Confusion)과 확산(Diffusion)

- 강력한 암호 알고리즘을 구축할 수 있는 두 가지 기본 연산(Claude Shannon)

  1. 혼돈(Confusion)
     - 평문과 암호문 사이의 상관관계를 숨기는 연산
     - 혼돈을 만족시키기 위한 일반적인 요소는 치환(substitution)
     - 예) DES, AES 등 현대 암호 시스템의 S-box 연산

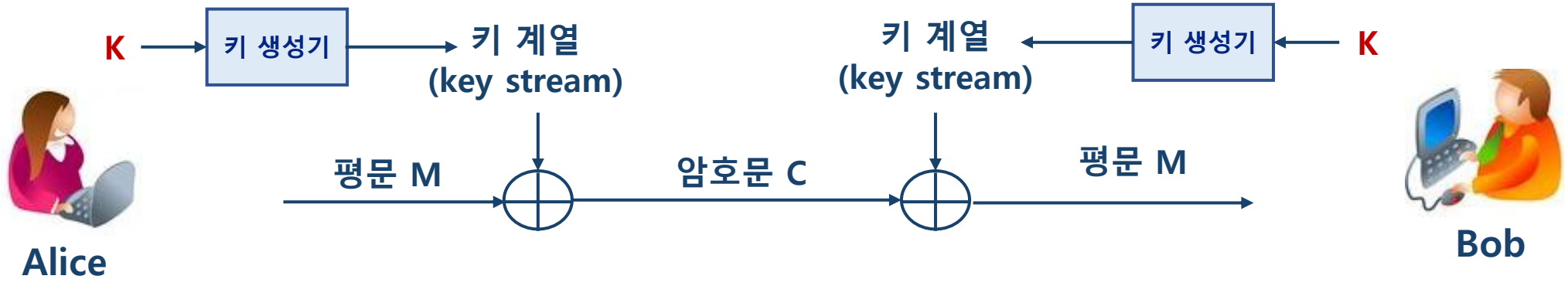
  2. 확산(Diffusion)
     - 평문의 통계적 특성을 숨기기 위한 목적
     - 하나의 평문 기호가 많은 암호문 기호에 영향을 미치도록하는 암호화 연산
     - 예) DES의 permutation이나 AES의 Mixcolumn 연산

- 혼동이나 확산 한가지만 수행하는 암호는 안전성을 제공하지 않음
- 두 연산을 혼합한 암호를 적 암호(*product ciphers*)라 함

3.2 스트림 암호

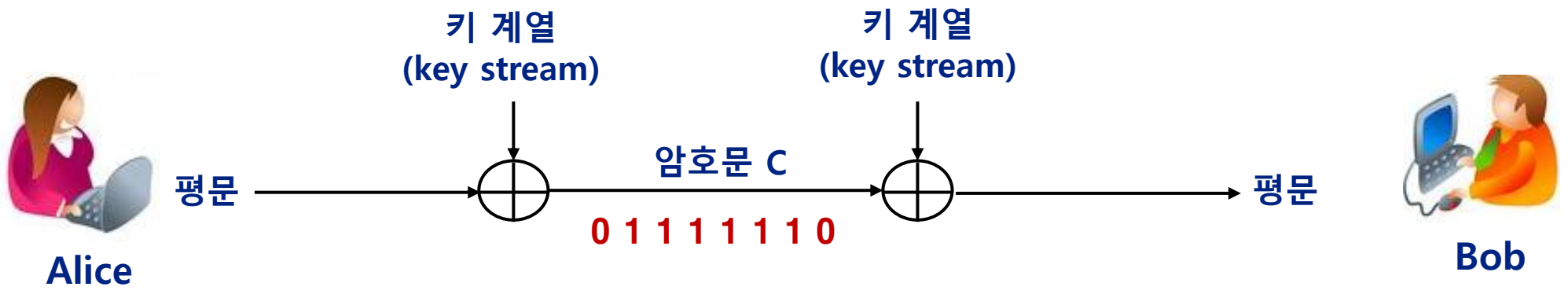
- 길이가 n비트인 키 K를 이용하여 긴 길이의 키 스트림(key stream)을 생성하고, 키 스트림과 평문 P를 XOR 연산을 하여 암호문 C 생성
- 동일한 키 스트림과 암호문 C를 XOR 연산을 수행하여 평문 P로 복호화
- 키 스트림을 사용하는 것은 일회성 암호를 실용적으로 일반화 한 것

스트림 암호의 암호화 : $C = M \oplus \text{Key stream}$
스트림 암호의 복호화 : $M = C \oplus \text{Key stream}$



3.2 스트림 암호

➢ 스트림 암호의 암호화/복호화 예



평문	1 0 1 1 1 0 1 1
키 계열	⊕ 1 1 0 0 0 1 0 1
암호문	0 1 1 1 1 1 1 0

암호문	0 1 1 1 1 1 1 0
키 계열	⊕ 1 1 0 0 0 1 0 1
평문	1 0 1 1 1 0 1 1

3.2 스트림 암호

➢ A5/1

- 유럽에서 사용하는 이동통신 표준 방식인 GSM(Global System for Mobile Communications)에서 기밀성 보장을 위해 사용됨
- 3개의 선형 피드백 시프트 레지스터(LFSR: Linear Feedback Shift Register) X, Y, Z 사용
  - 3개 레지스터의 전체 길이는 64비트
  - 키 길이도 64비트 이며, 키가 레지스터 초기값이 됨
  - 초기값 64비트를 이용하여 키 스트림 생성

X : 19비트(x_0, x_1, …, x_{18})
Y : 22비트(y_0, y_1, …, y_{21})
Z : 23비트(z_0, z_1, …, z_{22})

3.2 스트림 암호

➢ 레지스터 X 연산

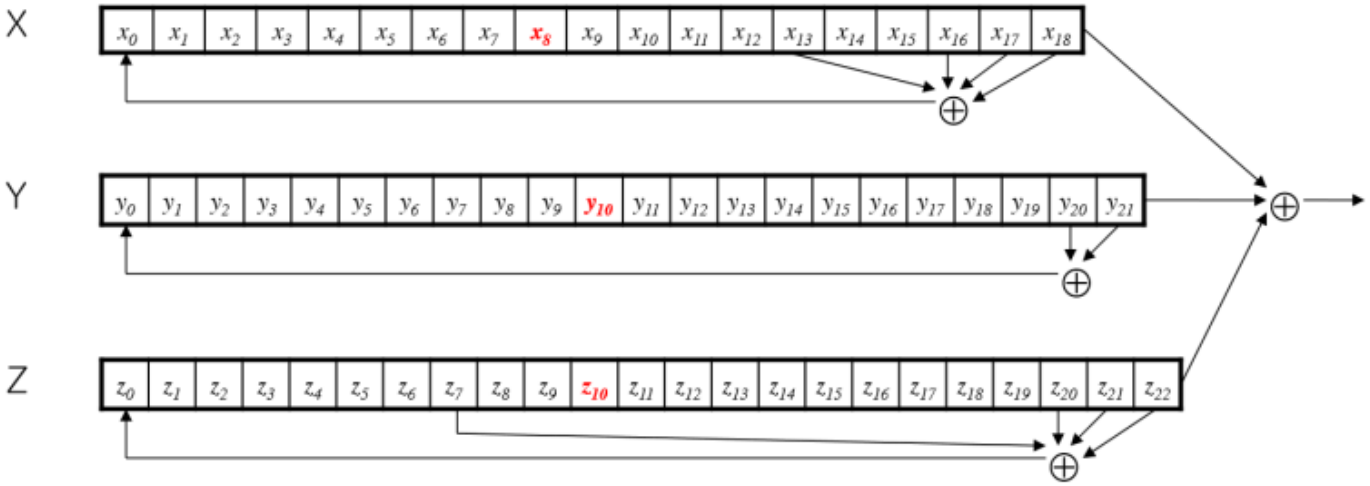
$$t = x_{13} \oplus x_{16} \oplus x_{17} \oplus x_{18}$$
$$x_i = x_{i-1} \text{ for}(i=18, 17, 16, \ldots 1)$$
$$x_0 = t$$

➢ 레지스터 Y 연산

$$t = y_{20} \oplus y_{21}$$
$$y_i = y_{i-1} \text{ for}(i=21, 20, \ldots 1)$$
$$y_0 = t$$

➢ 레지스터 Z 연산

$$t = z_7 \oplus z_{20} \oplus z_{21} \oplus z_{22}$$
$$z_i = z_{i-1} \text{ for}(i=22, 21, 20, \ldots 1)$$
$$z_0 = t$$



3.2 스트림 암호

➢ A1/5 키 스트림 생성과정

- 각각의 키 스트림 비트에 대해 다음을 수행

$$m = maj(x_8, y_{10}, z_{10})$$

  - maj(x, y, z) : 다수결 판별 함수로 x, y, z, 중 0이 다수면 0, 1이 다수면 1을 반환

- x, y, z가 다수인지 아닌 지에 따라 다음 단계 수행 여부 결정
  - x_8=m이면 X 단계 수행, y_{10}=m 이면 Y 단계 수행, z_{10}=m이면 Z 단계 수행

- 최종 키 스트림 비트는 $x_{18} \oplus y_{21} \oplus z_{22}$

3.2 스트림 암호

- 생성한 키 스트림 비트와 평문 비트를 XOR 연산하여 암호문 생성
  - 각 레지스터가 단계를 수행하게 되면 채워진 값이 변경되므로 키 스트림이 달라짐
  - 적어도 2개 이상의 레지스터 상태가 변함
- A1/5는 하드웨어로 구현이 쉽고 속도가 빠름

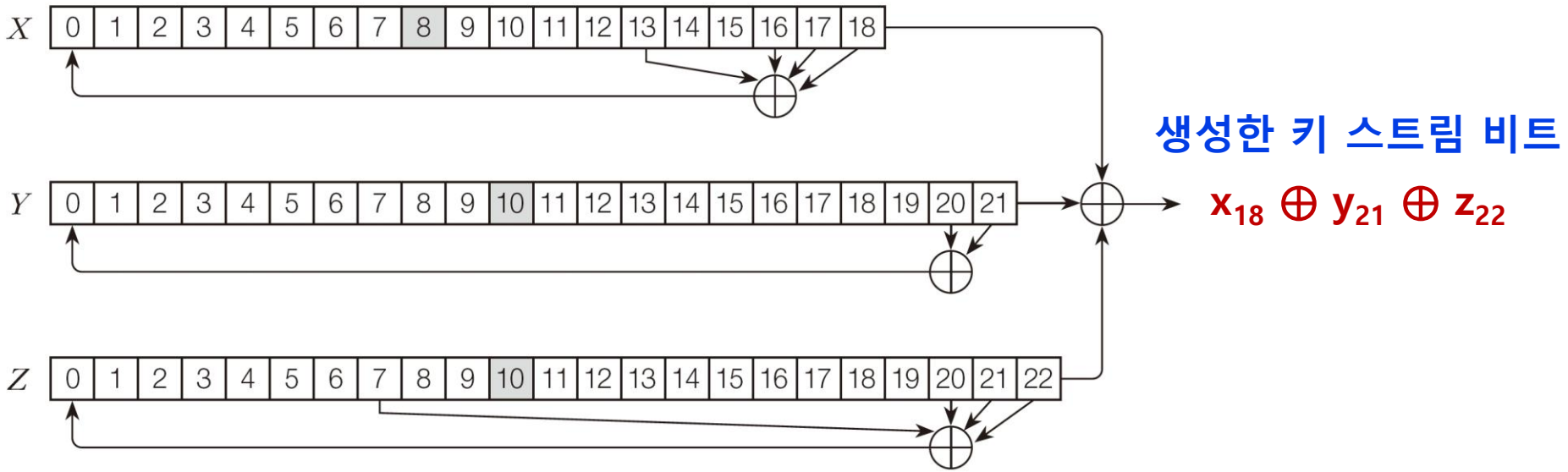
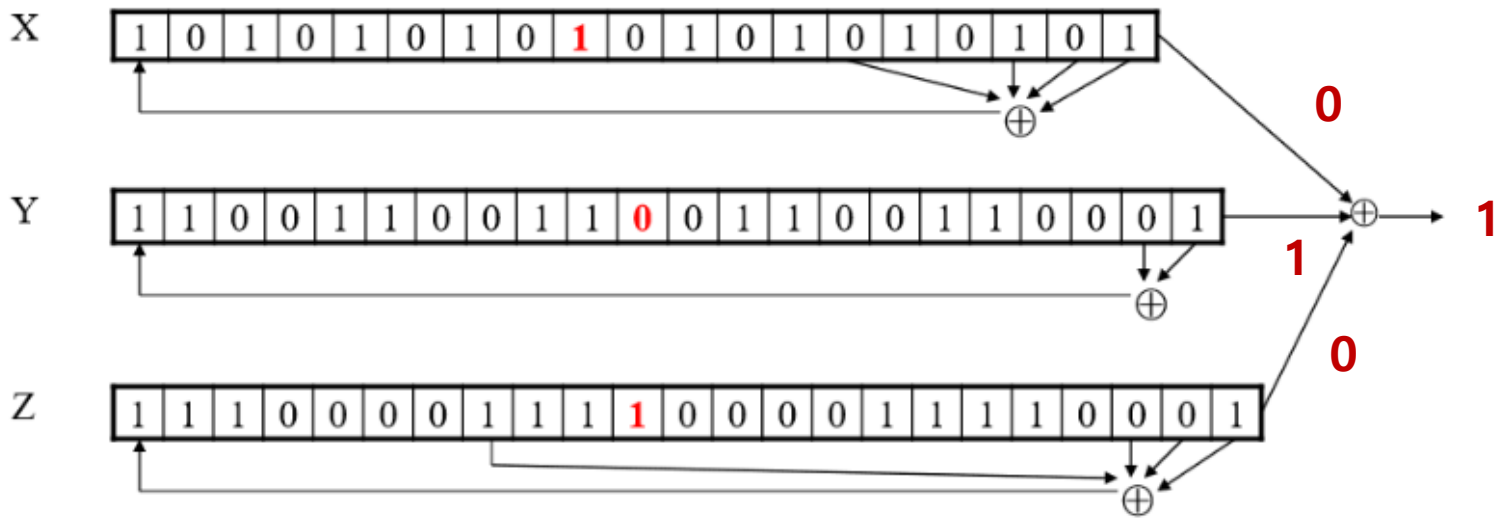


그림 3-1 A5/1 키 스트림 생성

3.2 스트림 암호

➢ A5/1 예제



- m = maj(x_8, y_{10}, z_{10}) = maj(1, 0, 1) = 1
- X와 Z는 실행되고 Y는 실행되지 않음
- 이때 생성되는 key stream의 비트는 0 ⊕ 1 ⊕ 0 =1

3.2 스트림 암호

➢ RC4

- Ron Rivest가 1987년 RSA Security에서 설계한 스트림 암호
- LFSR을 사용하지 않고, 바이트 단위로 작동
- 랜덤 치환 기법 사용
- 다양한 크기의 키 사용
- 소프트웨어 구현에 최적화
- TLS(Transport Layer Security), WEP(Wired Equivalent Privacy), WPA(Wi-Fi Protocol Access) 등 다양한 네트워크 보안 프로토콜에 사용됨
  - 현재는 여러 취약점이 발견되어 사용을 권장하지 않음

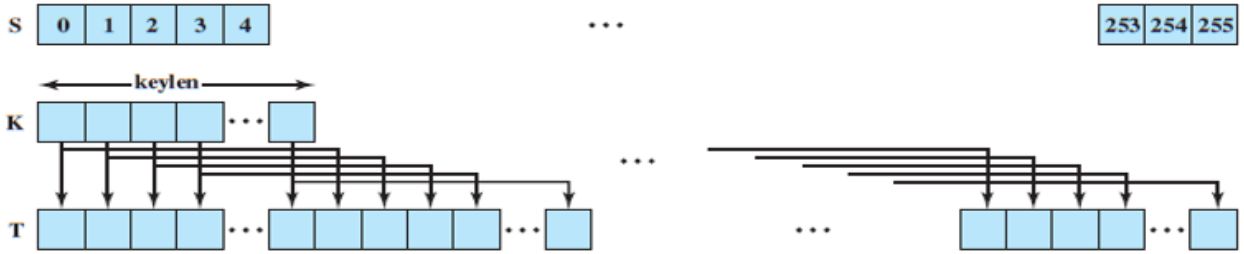
3.2 스트림 암호

➢ RC4 초기화

- 256바이트로 구성된 S[]의 한 바이트가 키 계열로 무작위로 선정
- S[ ], T[ ] 초기화

```
/* Initialization */
for i = 0 to 255 do
S[i] = i;
T[i] = K[i mod keylen];
```

- K : 키
- S[ ] : 0부터 255까지의 수를 올림차순으로 정렬
- T[ ] : 키 K를 길이만큼 반복하여 복사

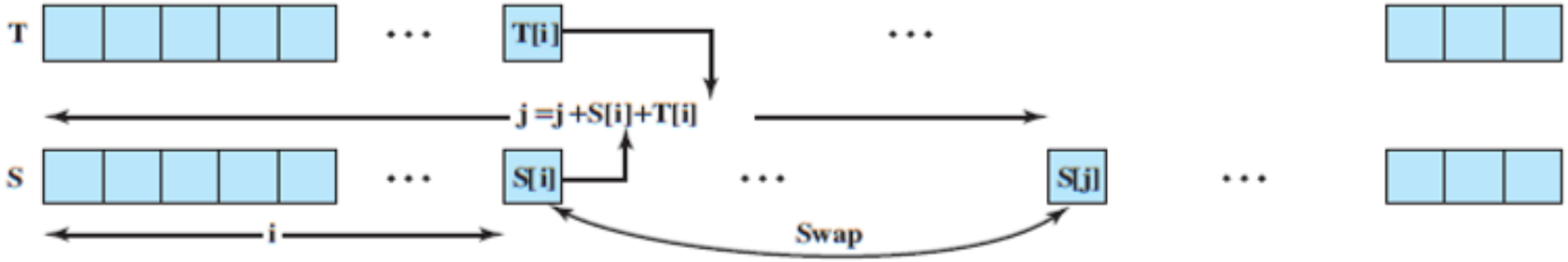


3.2 스트림 암호

➢ RC4 초기 전치

```
/* Initial Permutation of S */
j = 0;
for i = 0 to 255 do
        j = (j + S[i] +T[i]) mod 256;
        Swap (S[i], S[j]);
```

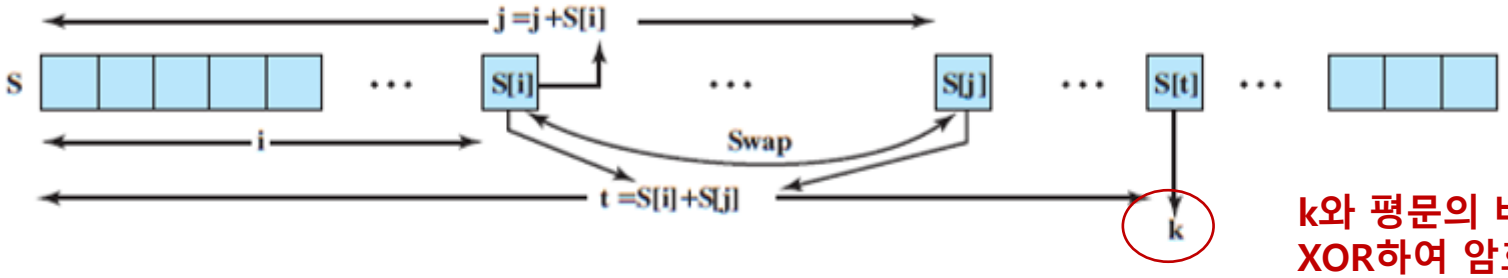
- S[i]를 T[i]의 값을 이용하여 S의 다른 바이트와 교환 수행



3.2 스트림 암호

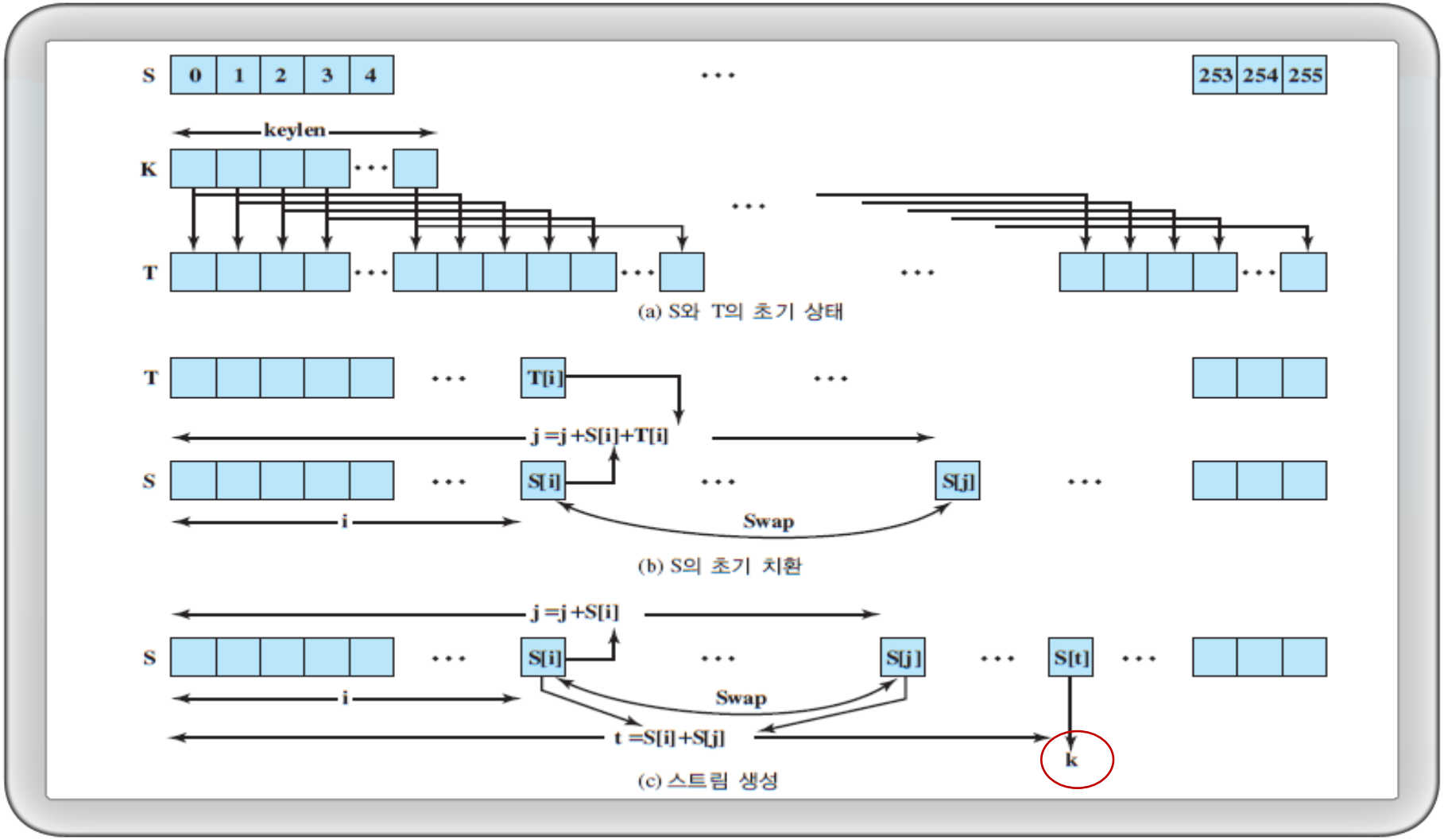
➢ RC4의 키 스트림 생성

```
/* Stream Generation */
i, j = 0
while (true)
        i = (i + 1) mod 256;
        j = (j + S[i]) mod 256;
        Swap (S[i], S[j]);
        t = (S[i] + S[j]) mod 256;
        k = S[t];
```



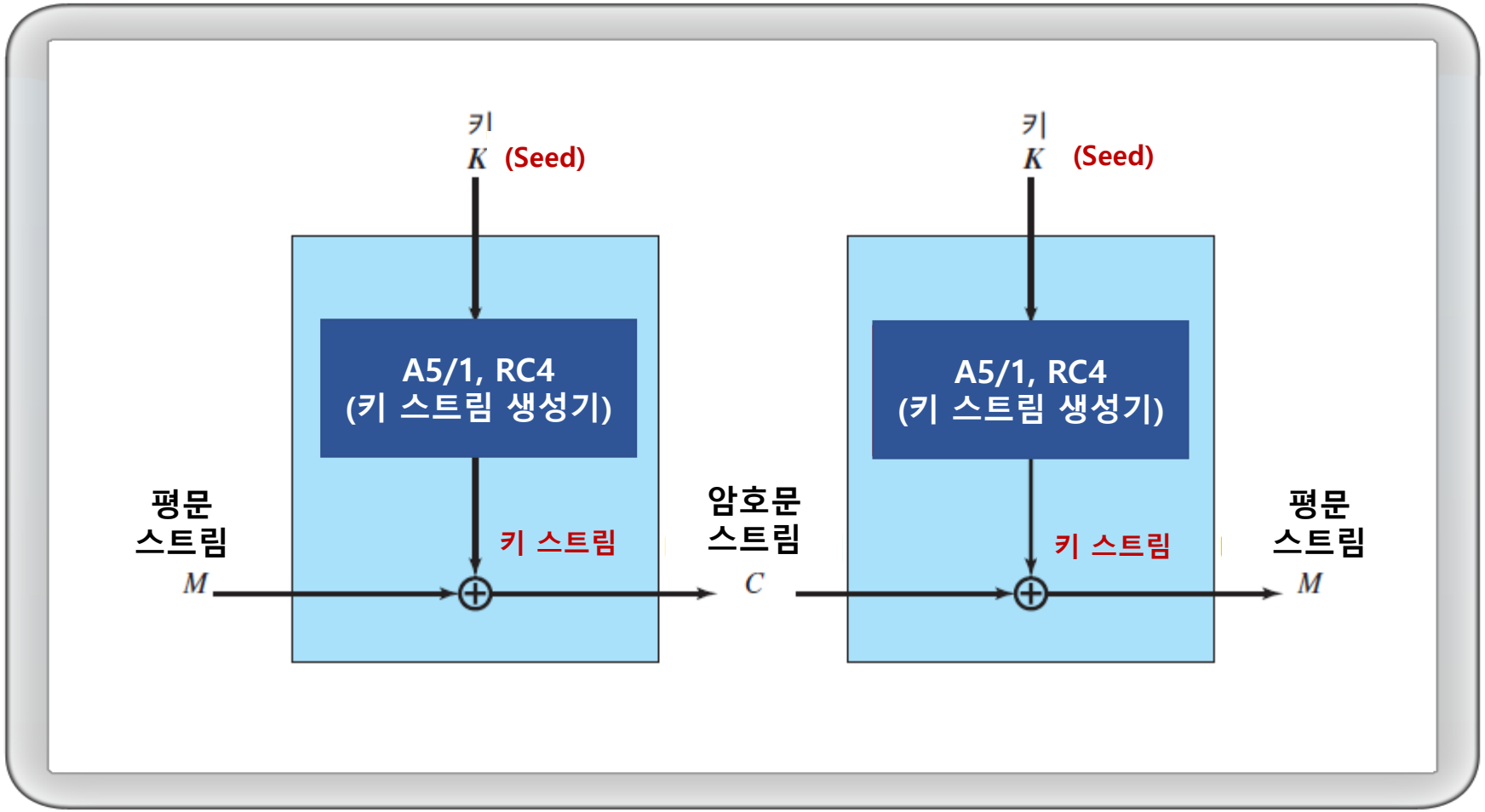
k와 평문의 바이트를
XOR하여 암호화 수행

3.2 스트림 암호



k와 평문의 바이트를
XOR하여 암호화 수행

3.2 스트림 암호


키
K (Seed)
A5/1, RC4
(키 스트림 생성기)
평문
스트림
M
키 스트림
암호문
스트림
C
키
K (Seed)
A5/1, RC4
(키 스트림 생성기)
키 스트림
평문
스트림
M

정리

- 스트림 암호는 전통적인 일회성 암호 시스템과 같은 방식으로 길이가 n비트인 키 K를 이용하여 긴 길이의 키 스트림(key stream)을 생성하고, 키 스트림과 평문 P를 XOR 연산을 하여 암호문 C 생성

- A5/1은 GSM에서 사용한 스트림 암호 시스템으로 선형 피드백 시프트 레지스터(LFSR)를 이용하여 비트 단위로 키 스트림을 생성하여 암호화
  - 2G 이동통신의 음성 암호화에 사용하였으나, 64비트의 짧은 키 길이와 여러가지 취약점이 발견됨

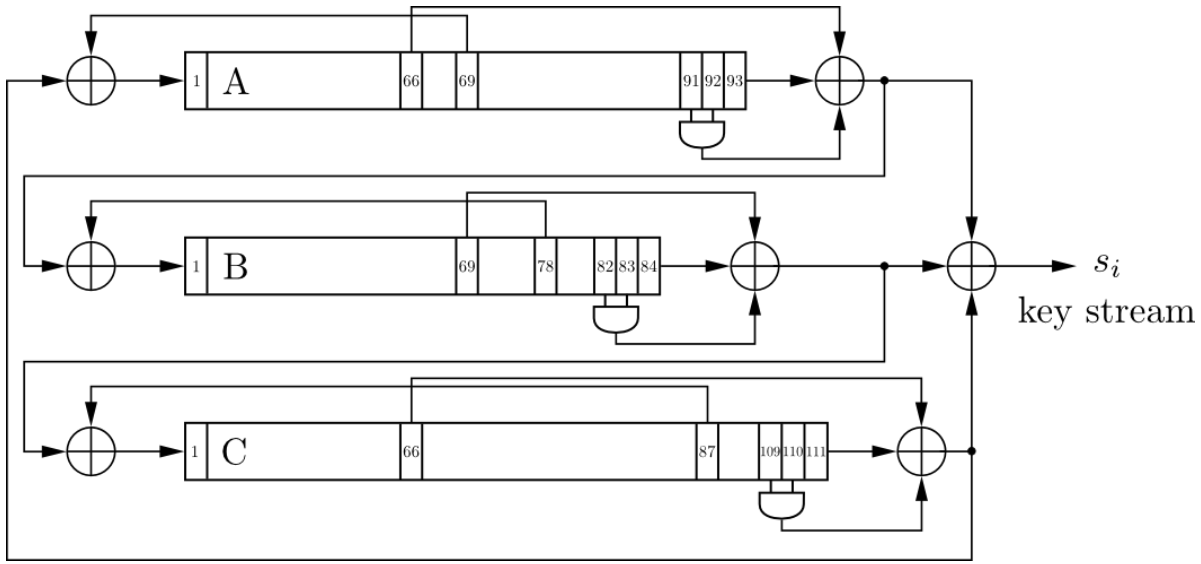
- Ron Rivest가 개발한 RC4는 LFSR을 사용하지 않고, 랜덤 치환 기법을 이용하여 바이트 단위의 키 스트림 생성
  - 다양한 네트워크 보안 프로토콜에서 사용하였으나 현재는 여러 취약점이 발견되어 사용을 권장하지 않음

참고 - Trivium

- 2004년부터 2008년까지 유럽의 암호 전문가 단체인 ECRYPT(Europian Network of Excellent in Cryptography)의 eStream 프로젝트에서 개발
- 하드웨어 기반으로 구현하기에 용이
- 80비트의 키를 사용하는 스트림 암호 시스템
- 3개의 시프트 레지스터를 결합하여 설계
  - 길이가 93, 84, 111인 3개의 비선형 피드백 시프트 레지스터(NLFSR) 사용

Trivium

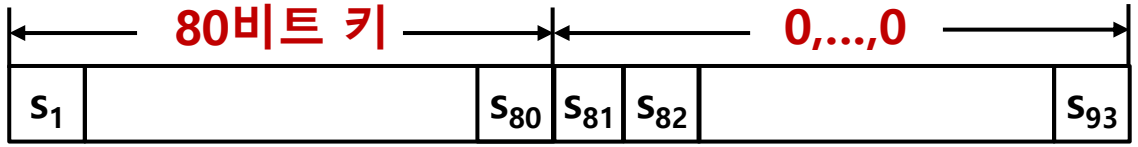
- 3개의 시프트 레지스터 결과의 XOR를 이용하여 키 스트림 s_i 를 생성
- 하드웨어 규모가 작음
  - 총 레지스터 개수: 288
  - 비-선형성: 3개의 AND-게이트
  - 7개의 XOR-게이트 (이 중 4개는 3개의 입력을 가짐)



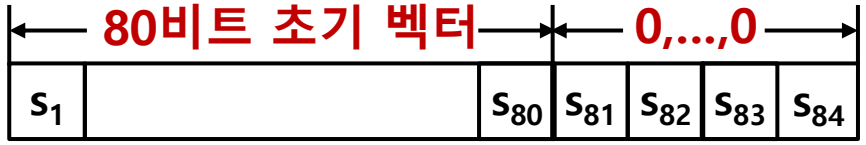
Trivium

➢ 초기화

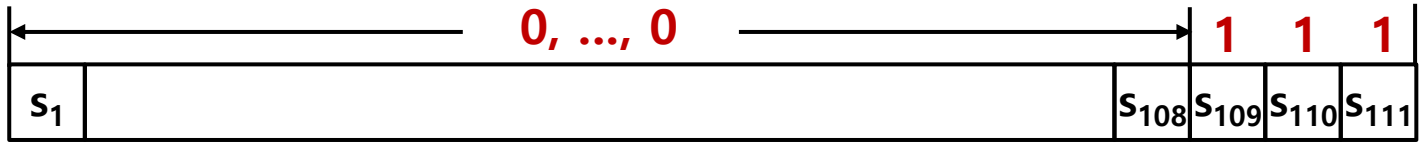
- 80 비트의 키와 80 비트의 초기벡터(IV, Initial Vector)를 93비트 레지스터와 84비트 레지스터에 로딩
- 111비트 레지스터의 마지막 3비트는 1로 그 외 모든 비트는 0으로 설정



93비트 레지스터



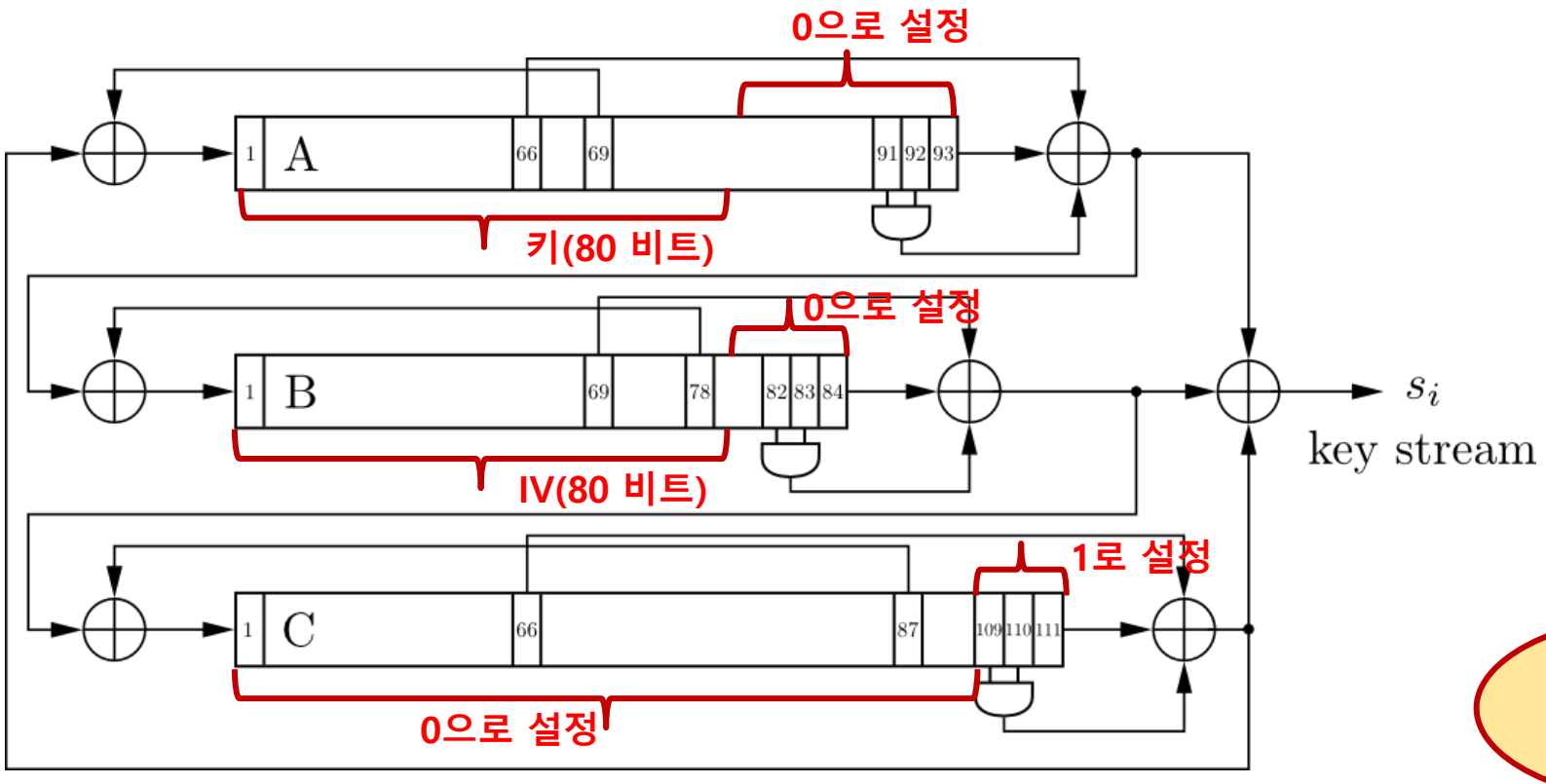
84비트 레지스터



111비트 레지스터

Trivium

➢ 초기화



IV의 역할은?

※ IV(Initial vector) : 비밀 정보는 아니며, 암호화 세션마다 서로 다른 값을 가짐
- 한번 쓰이는 수(number used once)라는 의미로 nonce라 함

Trivium

➢ 준비

- 결과를 생성하지 않고 4 x 288 = 1152번의 클록 암호화 수행

➢ 암호화

- 1153번째 출력 비트로 시작하는 키 스트림 구성
- 3개의 NLFSR 결과의 XOR 합이 키 스트림 s_i 를 생성
- 클록 사이클당 최대 64 비트 결과까지 생성하기 위해 병렬적으로 설계될 수 있음

	레지스터 길이	피드백 비트	비트 포워드 비트	AND 입력
A	93	69	66	91, 92
B	84	78	69	82, 83
C	111	87	66	109, 110

Trivium

- Trivium은 하드웨어로 구현하기에 적합
  - 288비트의 시프트 레지스터와 부울 연산으로 구성
  - 하드웨어로 구현 시, 병렬화 정도에 따라 3500개에서 5500개 게이트로 구성

- AES와 같은 대부분의 블록 암호보다 하드웨어 구현이 간단하고 매우 빠름
  - IoT 환경 등 경량 암호가 필요한 분야에서 많이 연구됨
- 아직까지는 무차별 대입 공격 이외의 더 나은 공격 방법이 알려져 있지는 않음
- 표준으로 채택되지 않아 연구용으로 주로 사용

참고

➢ ChaCha20 스트림 암호

- 개발자: Daniel J. Bernstein (2008년, Salsa20 암호 방식을 개선)
- 용도: TLS 1.3, OpenSSH, WireGuard VPN, 구글 Chrome 등

- 키 길이: 256비트(32바이트), Nonce: 96비트(12바이트)
  - 같은 키+nonce 조합을 절대 재사용하면 안 됨
- 라운드 수: 20라운드(라운드 수에 따라 ChaCha8, ChaCha12 등도 존재)

- 보안성: 현재까지 알려진 실질적인 공격 없음
- 성능: 하드웨어뿐 아니라 소프트웨어(특히 모바일 CPU)에서도 빠르게 동작
- 현재 인터넷 보안 환경에서 RC4를 대체하는 사실상의 표준 스트림 암호